Finite Element Methods

Theory, Geometry, and Computation

Aldebert Lecornu

École Polytechnique de Louvain
Université catholique de Louvain

February 18, 2026

Contents

Chapter 1

Finite Element Methods in One Spatial Dimension

This chapter is devoted to the finite element method (FEM) in one spatial dimension. Although the spatial domain is restricted to an interval, this setting already contains all the essential ingredients of the method: mesh generation, finite-dimensional approximation spaces, variational formulations, and the connection between functional analysis and numerical discretization.

Throughout this chapter, we use two model problems: the Poisson equation and the Euler–Bernoulli beam equation. These problems are both elliptic, 2nd and 4th order. They naturally lead to different regularity requirements on the solution space.

1.1 Model problems in one spatial dimension

Let $\Omega = (0, L) \subset \mathbb{R}$ be a one-dimensional spatial domain.

Poisson equation (second-order elliptic). The Poisson problem reads

$$-\frac{d}{dx}\left(\kappa(x)\,\frac{du}{dx}\right) = f(x) \qquad \text{in } \Omega, \tag{1.1}$$

with appropriate boundary conditions. This equation will serve as the primary introduction to the finite element method, as its variational formulation involves only first derivatives and leads to the simplest finite element spaces.

Euler–Bernoulli beam equation (fourth-order elliptic). The flexion of an elastic beam is governed by

$$\frac{d^2}{dx^2}\left(EI(x)\,\frac{d^2u}{dx^2}\right) = q(x) \qquad \text{in } \Omega, \tag{1.2}$$

where $u(x)$ denotes the transverse displacement and $q(x)$ denotes the force per unit of lengths applied transversally to the beam. This fourth-order problem requires higher regularity of the solution and motivates the introduction of finite element spaces with enhanced continuity properties.

1.2 Finite Elements in one dimension

The starting point of the finite element method is *a mesh*. In dimension one, it is a subdivision of the interval $[0, L]$ into a finite number of subintervals (see Figure 1.1). A one-dimensional mesh is defined as a partition

$$0 = x_0 < x_1 < \cdots < x_N = L,$$

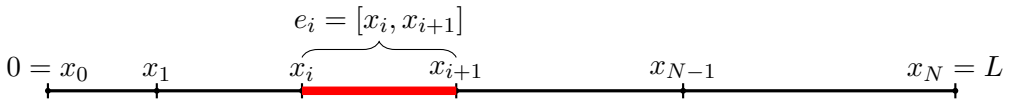

Figure 1.1: A one-dimensional mesh of $[0, L]$ and one element $e_i = [x_i, x_{i+1}]$.

where each subinterval

$$e_i = [x_i, x_{i+1}]$$

is called an element (not a finite element!). The extremities x_i and x_{i+1} are the called the vertices of the element. Although trivial in appearance, this construction already introduces the key geometric structure on which the finite element spaces are built.

In the sense of Pierre-Arnaud Raviart [1], a finite element is primarily an interpolation tool designed to approximate fields by functions belonging to a finite-dimensional space. The *definition of a finite element* as a triplet introduced by Raviart, formalizes this idea by separating geometry, approximation, and interpolation. The triplet is

$$(K,\ \mathcal{P}_K,\ \Sigma_K),$$

where:

- K is a simple geometric entity (in the 1D case, the entity K is an interval K_i);
- $\mathcal{P}_K$ is a finite-dimensional space of functions defined on K;
- Σ_K is a set of linearly independent degrees of freedom, forming a basis of the dual space $\mathcal{P}'_K$.

This definition separates geometry, approximation, and interpolation, and provides a systematic way to construct finite-dimensional spaces. In order to make this formal definition more concrete, we will define four 1D finite elements all based on the same geometric entity.

1.2.1 The one-dimensional P_1 finite element

We start with the simplest (and most fundamental) conforming finite element in one space dimension: the continuous, piecewise affine element. In the standard notation, it is called the P_1 element.

The finite element triplet. Let $K = [x_i, x_{i+1}] \subset \mathbb{R}$. The one-dimensional P_1 finite element is defined by the triplet

$$(K, \mathbb{P}_1(K), \Sigma_K),$$

where:

- the *geometric entity* is the closed interval K;
- the *local space* is

$$\mathbb{P}_1(K) = \{v : K \to \mathbb{R} \mid v(x) = a + bx,\ a, b \in \mathbb{R}\},$$

  a vector space of dimension 2;

- the *degrees of freedom* are the endpoint evaluations

$$\Sigma_K = \{\ell_0, \ell_1\}, \qquad \ell_0(v) = v(x_i), \quad \ell_1(v) = v(x_{i+1}).$$

Unisolvence and interpolation. For any $(\alpha, \beta) \in \mathbb{R}^2$, there exists a unique $v_h \in \mathbb{P}_1(K)$ such that $v_h(x_i) = \alpha$ and $v_h(x_{i+1}) = \beta$. This property (unisolvence) implies the existence of the local interpolation operator

$$\mathcal{I}_K : C^0(K) \to \mathbb{P}_1(K), \qquad (\mathcal{I}_K v)(x_i) = v(x_i), \quad (\mathcal{I}_K v)(x_{i+1}) = v(x_{i+1}).$$

Local Lagrange basis. The basis functions $\{\varphi_0, \varphi_1\} \subset \mathbb{P}_1(K)$ associated with Σ_K are characterized by (see Figure 1.2)

$$\varphi_m(x_{i+n}) = \delta_{mn} \qquad (m, n \in \{0, 1\}),$$

and are given explicitly by

$$\varphi_0(x) = \frac{x_{i+1} - x}{x_{i+1} - x_i}, \qquad \varphi_1(x) = \frac{x - x_i}{x_{i+1} - x_i}.$$

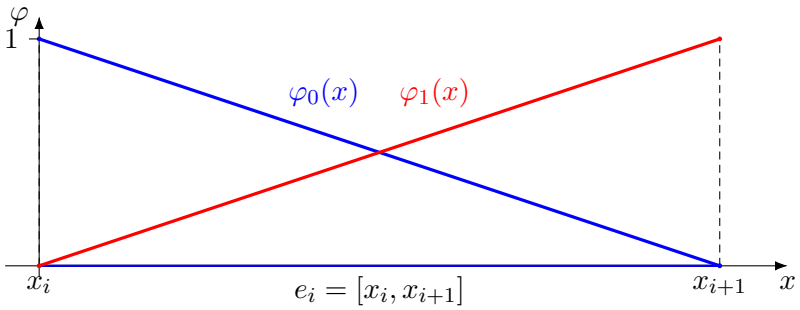

Figure 1.2: Local Lagrange basis functions $\varphi_0, \varphi_1 \in \mathbb{P}_1(K)$ on the element $e_i = [x_i, x_{i+1}]$.

Global P_1 space. Over a mesh $\mathcal{T}_h$, the corresponding global finite element space is

$$V_h^{(1)} = \{v_h \in C^0([0, L]) \mid v_h|_K \in \mathbb{P}_1(K) \ \forall K \in \mathcal{T}_h\},$$

with essential boundary conditions enforced by restricting to $V_h^{(1)} \cap H_0^1(0, L)$ when needed. The functions in $V_h^{(1)}$ are globally continuous and piecewise affine.

Figure 1.3 shows the interpolation of a simple function using P_1 elements on a uniform mesh of 10 1D intervals. One sees that the interpolation is continuous but not derivable at vertices and that the total number of degrees of freedom for the interpolation is equal to the number of vertices of the mesh.

1.2.2 The one-dimensional P_k Lagrange finite element (equispaced nodes)

The P_1 element generalizes naturally to higher polynomial degree. In one dimension, the most common family is the *Lagrange* family of order k, built from point values at $k + 1$ nodes on the element.

Reference element and equispaced nodes. In the P_1 case, we have defined the basis functions (also named shape functions) in the physical space x. They actually involve explicitly the coordinates x_i of a specific element K_i. In a more general case of high order elements or elements of higher dimensions (triangles in 2D, tetrahedra in 3D), it is convenient to define a reference space for the element. Let the reference interval be $\widehat{K} = [0, 1]$. For a fixed integer $k \geq 1$, define the equispaced nodes

$$\widehat{\xi}_j := \frac{j}{k}, \qquad j = 0, 1, \ldots, k.$$

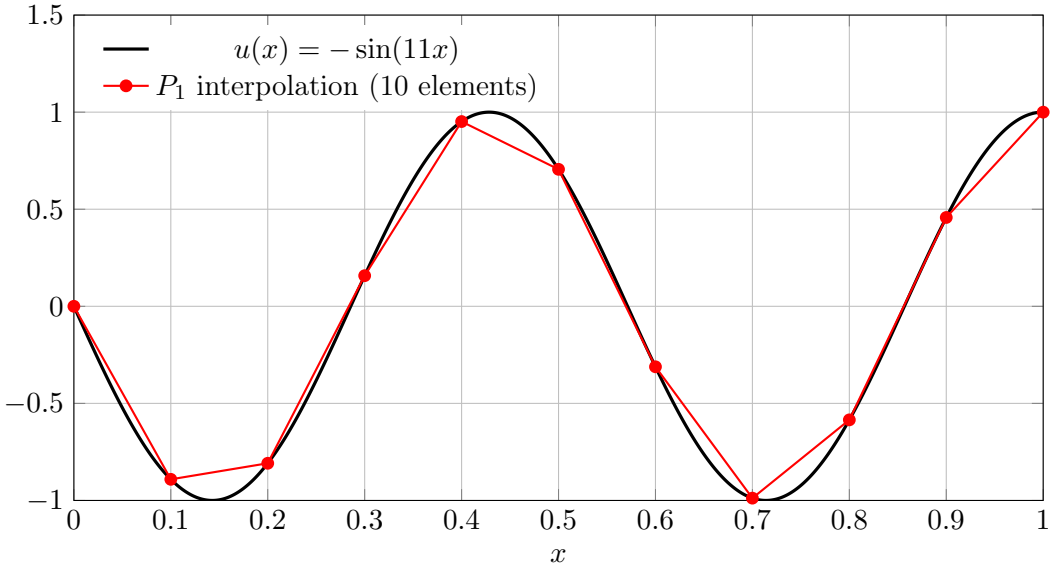

Figure 1.3: Piecewise linear (P_1) interpolation of $u(x) = -\sin(11x)$ on $[0,1]$ using 10 uniform elements. Red bullets indicate the nodal degrees of freedom.

The Lagrange shape functions $\{\widehat{\varphi}_j\}_{j=0}^k \subset \mathbb{P}_k(\widehat{K})$ are defined by the interpolation conditions

$$\widehat{\varphi}_j(\widehat{\xi}_m) = \delta_{jm} \qquad (j,m = 0,\dots,k).$$

They admit the explicit product formula

$$\widehat{\varphi}_j(\xi) = \prod_{\substack{m=0 \\ m\neq j}}^{k} \frac{\xi - \widehat{\xi}_m}{\widehat{\xi}_j - \widehat{\xi}_m}, \qquad \xi \in [0,1]. \tag{1.3}$$

Mapped element. Let $K = [x_i, x_{i+1}]$. The standard affine mapping

$$F_K : \widehat{K} \to K, \qquad x = F_K(\xi) = x_i + h_K\,\xi, \qquad h_K := x_{i+1} - x_i,$$

transports the reference nodes to the physical nodes

$$x_{i,j} := F_K(\widehat{\xi}_j) = x_i + \frac{j}{k} h_K, \qquad j = 0,\dots,k.$$

The local shape functions on K are defined by pullback:

$$\varphi_{K,j}(x) := \widehat{\varphi}_j(F_K^{-1}(x)) = \widehat{\varphi}_j\left(\frac{x - x_i}{h_K}\right).$$

Finite element triplet. The one-dimensional P_k Lagrange element with equispaced nodes is the triplet

$$(K, \mathbb{P}_k(K), \Sigma_K),$$

where the degrees of freedom are the nodal evaluations at the $k+1$ points $\{x_{i,j}\}_{j=0}^k$:

$$\Sigma_K = \{\ell_0,\dots,\ell_k\}, \qquad \ell_j(v) = v(x_{i,j}).$$

The associated local interpolation operator is

$$\mathcal{I}_K : C^0(K) \to \mathbb{P}_k(K), \qquad (\mathcal{I}_K v)(x_{i,j}) = v(x_{i,j}), \quad j = 0,\dots,k,$$

and the local Lagrange basis $\{\varphi_{K,j}\}_{j=0}^k$ satisfies

$$\varphi_{K,j}(x_{i,m}) = \delta_{jm}.$$

Global P_k space. Over a mesh $\mathcal{T}_h$, the corresponding conforming finite element space is

$$V_h^{(k)} = \{v_h \in C^0([0,L]) \mid v_h|_K \in \mathbb{P}_k(K)\ \forall K \in \mathcal{T}_h\}.$$

This space is a natural discrete subspace of $H^1(0,L)$, and increasing k enhances the approximation power whenever the exact solution is sufficiently smooth.

Cubic Lagrange shape functions (P_3) on $[0,1]$. We give here the example of the Lagrange P_3 equidistant element shape functions in Figure 1.4.

The four nodal points are

$$\xi_0 = 0, \qquad \xi_1 = \frac{1}{3}, \qquad \xi_2 = \frac{2}{3}, \qquad \xi_3 = 1.$$

Now is probably the good time to distinguish between the concept of vertices and the concept of nodes. The two vertices are the extremity of the element (segment in 1D). The P^3 finite element approximation requires however four nodes, some whose positions coincide with vertices and some which are internal to the element. Nodes of the first category are sometimes called vertex-node whereas nodes of the second, internal-nodes. So to summarize, a 1D element has always 2 vertices whereas a 1D finite element may have many nodes.

A cubic Lagrange shape functions is associated to each of the four nodes

$$\begin{aligned}
\varphi_0(\xi) &= \frac{(\xi-\frac{1}{3})(\xi-\frac{2}{3})(\xi-1)}{(0-\frac{1}{3})(0-\frac{2}{3})(0-1)} \\
&= -\frac{9}{2}\xi^3 + \frac{27}{2}\xi^2 - 9\xi + 1,
\end{aligned}$$

$$\begin{aligned}
\varphi_1(\xi) &= \frac{\xi(\xi-\frac{2}{3})(\xi-1)}{(\frac{1}{3})(\frac{1}{3}-\frac{2}{3})(\frac{1}{3}-1)} \\
&= \frac{27}{2}\xi^3 - 18\xi^2 + \frac{9}{2}\xi,
\end{aligned}$$

$$\begin{aligned}
\varphi_2(\xi) &= \frac{\xi(\xi-\frac{1}{3})(\xi-1)}{(\frac{2}{3})(\frac{2}{3}-\frac{1}{3})(\frac{2}{3}-1)} \\
&= -\frac{27}{2}\xi^3 + \frac{45}{2}\xi^2 - 9\xi,
\end{aligned}$$

$$\begin{aligned}
\varphi_3(\xi) &= \frac{\xi(\xi-\frac{1}{3})(\xi-\frac{2}{3})}{(1)(1-\frac{1}{3})(1-\frac{2}{3})} \\
&= \frac{9}{2}\xi^3 - \frac{9}{2}\xi^2 + \xi.
\end{aligned}$$

Figure 1.5 shows the interpolation of a simple function using P_3 elements on a uniform mesh of 4 1D intervals. The interpolation is continuous but not derivable at vertices and that the total number of degrees of freedom for the interpolation is equal to the number of vertices of the mesh plus the number of internal nodes of every element – in our case, 5 vertices, 4 elements that have 2 internal nodes for a total of $5 + 4 * 2 = 13$ degrees of freedom.

Remark (about equispaced nodes). Equispaced Lagrange nodes are convenient for exposition, but they are not always the best choice numerically for high orders because of conditioning and oscillations. In practice, one often prefers Gauss–Lobatto or other optimized node distributions. In this chapter, we use equispaced nodes for simplicity and because they match the basic intuition behind nodal interpolation.

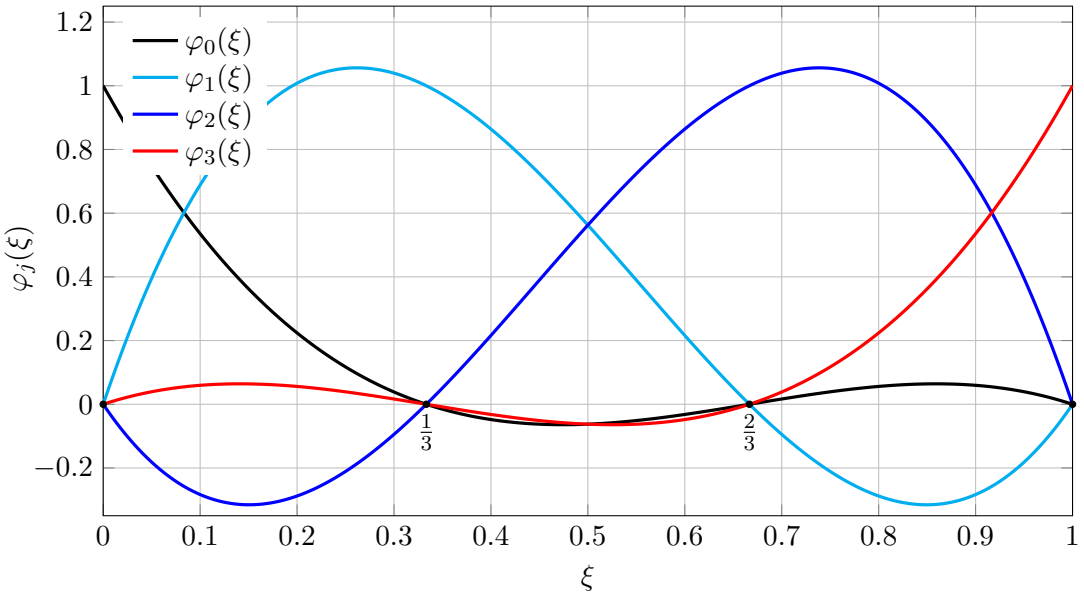

Figure 1.4: Local Lagrange P_3 basis functions on the reference element $[0, 1]$, with nodes $\{0, \frac{1}{3}, \frac{2}{3}, 1\}$.

1.2.3 The one-dimensional Hermite finite element

We now introduce the classical one-dimensional Hermite finite element. Unlike the P_1 element, which enforces only continuity of the function, Hermite elements are designed to ensure continuity of both the function and its first derivative. They are therefore well suited for fourth-order problems such as the Euler–Bernoulli beam equation.

Geometric entity. Let $K = [x_i, x_{i+1}] \subset \mathbb{R}$ be a closed interval. As before, the endpoints x_i and x_{i+1} are the vertices of the element.

Local space of shape functions. The local approximation space is defined as

$$\mathcal{P}_K = \mathbb{P}_3(K),$$

the space of polynomials of degree at most three on K. This is a vector space of dimension four.

Degrees of freedom. The degrees of freedom are chosen as the values of the function and its first derivative at the endpoints:

$$\Sigma_K = \{\ell_1, \ell_2, \ell_3, \ell_4\},$$

with

$$\ell_1(v) = v(x_i), \qquad \ell_2(v) = v'(x_i), \qquad \ell_3(v) = v(x_{i+1}), \qquad \ell_4(v) = v'(x_{i+1}).$$

These linear functionals are linearly independent and form a basis of the dual space $\mathcal{P}_K'$.

Unisolvence and interpolation. The triplet $(K, \mathcal{P}_K, \Sigma_K)$ is unisolvent: for any prescribed values of v and v' at the endpoints, there exists a unique polynomial $v_h \in \mathcal{P}_K$ satisfying the four degrees of freedom.

As a consequence, the element defines a local interpolation operator

$$\mathcal{I}_K : C^1(K) \longrightarrow \mathcal{P}_K,$$

such that

$$\ell_j(\mathcal{I}_K v) = \ell_j(v) \qquad \text{for } j = 1, \ldots, 4.$$

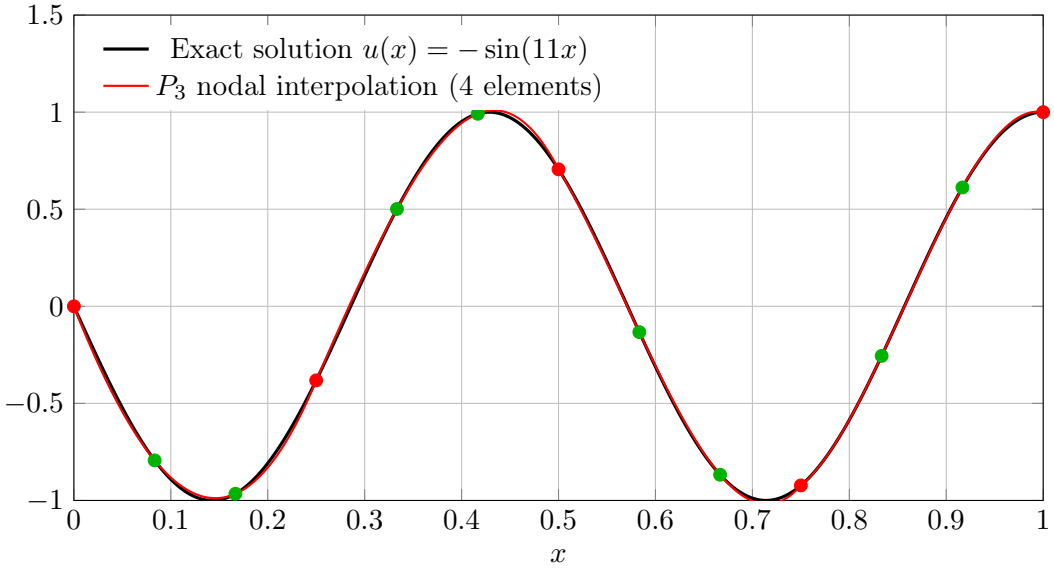

Figure 1.5: P_3 nodal (Lagrange) interpolation of $u(x) = -\sin(11x)$ on $[0, 1]$ using four uniform elements. Shared inter-element nodes are shown in red, while interior element nodes are shown in green.

Local basis functions. Let $h = x_{i+1} - x_i$ and introduce the local coordinate $\xi = (x - x_i)/h \in [0, 1]$. The basis functions associated with the degrees of freedom are the Hermite shape functions:

$$\begin{aligned}
\psi_0(\xi) &= 1 - 3\xi^2 + 2\xi^3, \\
\psi_1(\xi) &= h\left(\xi - 2\xi^2 + \xi^3\right), \\
\psi_2(\xi) &= 3\xi^2 - 2\xi^3, \\
\psi_3(\xi) &= h\left(-\xi^2 + \xi^3\right).
\end{aligned}$$

They satisfy

$$\begin{aligned}
\psi_0(0) = 1,\ \psi_0'(0) = 0, \psi_0(1) = 0,\ \psi_0'(1) = 0, \\
\psi_1(0) = 0,\ \psi_1'(0) = 1, \psi_1(1) = 0,\ \psi_1'(1) = 0, \\
\psi_2(0) = 0,\ \psi_2'(0) = 0, \psi_2(1) = 1,\ \psi_2'(1) = 0, \\
\psi_3(0) = 0,\ \psi_3'(0) = 0, \psi_3(1) = 0,\ \psi_3'(1) = 1
\end{aligned}$$

where derivatives are

$$\psi'(\xi(x)) = \frac{d\psi}{dx} = \frac{d\psi}{d\xi}\frac{1}{h}$$

as in Figure 1.6.

Regularity. Functions in $\mathcal{P}_K$ are continuously differentiable on K. When assembled over a mesh, the resulting global finite element space consists of piecewise cubic functions that are globally C^1-continuous. This makes Hermite elements natural discrete subspaces of $H^2(\Omega)$, which is required for fourth-order variational problems. Figure 1.7 shows an example of a finite element interpolation using cubic hermite elements.

1.2.4 A discontinuous P_1 finite element based on Legendre polynomials

We now introduce a fourth type of finite element, fundamentally different from the nodal and Hermite elements discussed so far: a discontinuous P_1 finite element. In this case, continuity across elements is not enforced, and each element carries its own independent degrees of freedom.

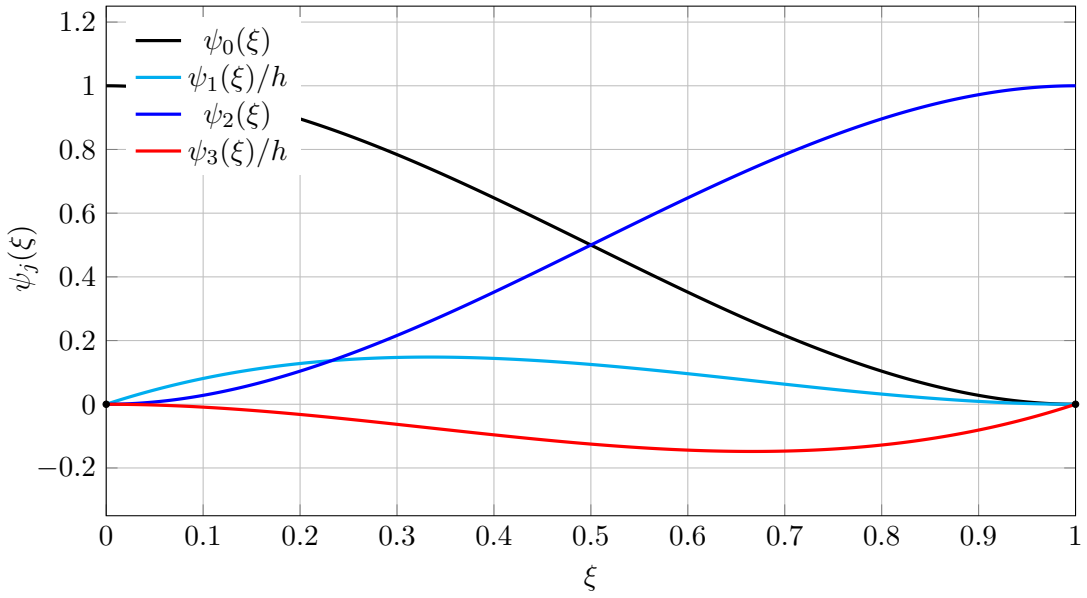

Figure 1.6: Cubic Hermite basis functions on the reference element $[0, 1]$.

The purpose of this element is not nodal interpolation, but rather the approximation of a function through its *average value* and its *average slope* on each element. This construction is naturally expressed using Legendre polynomials.

Geometric entity. Let $K = [x_i, x_{i+1}] \subset \mathbb{R}$ be an interval of length $h = x_{i+1} - x_i$. No continuity constraints are imposed between neighboring elements.

Local space of shape functions. We define the local approximation space as

$$\mathcal{P}_K = \mathbb{P}_1(K),$$

the space of polynomials of degree at most one on K. Rather than using the monomial basis $\{1, x\}$, we choose a basis inspired by the first two Legendre polynomials.

Introducing the local coordinate

$$\xi = \frac{2(x - x_i)}{h} - 1 \in [-1, 1],$$

the basis functions are defined as

$$\phi_0(\xi) = 1, \qquad \phi_1(\xi) = \xi.$$

These functions are orthogonal with respect to the L^2 inner product on $[-1, 1]$, and ϕ_1 has zero mean.

Degrees of freedom. The degrees of freedom are defined as linear functionals acting on a sufficiently regular function v:

$$\Sigma_K = \{\ell_0, \ell_1\},$$

with

$$\ell_0(v) = \frac{1}{h} \int_K v(x)\, dx,$$
$$\ell_1(v) = \frac{1}{h} \int_K v'(x)\, dx.$$

The first degree of freedom represents the *mean value* of the function on the element. The second degree of freedom represents the *mean slope* of the function on the element.

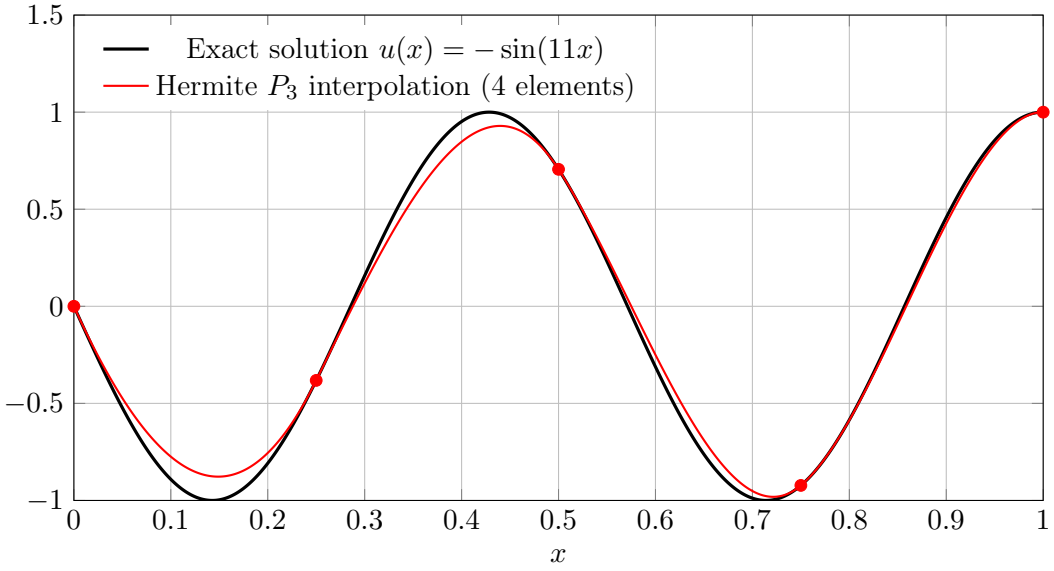

Figure 1.7: Hermite P_3 interpolation (piecewise cubic, globally C^1) of $u(x) = -\sin(11x)$ on $[0, 1]$ using four uniform elements. The exact solution is shown in black; the Hermite interpolant is shown in red.

Interpolation. The associated interpolation operator

$$\mathcal{I}_K : C^1(K) \longrightarrow \mathcal{P}_K$$

is defined by matching these two moments. The interpolant can be written explicitly as

$$(\mathcal{I}_K v)(x) = \ell_0(v)\,\phi_0(\xi) + \frac{h}{2}\,\ell_1(v)\,\phi_1(\xi),$$

where the scaling ensures that the derivative of the interpolant has the correct average value on K.

Properties. By construction,

$$\frac{1}{h}\int_K \mathcal{I}_K v\,dx = \frac{1}{h}\int_K v\,dx, \qquad \frac{1}{h}\int_K (\mathcal{I}_K v)'\,dx = \frac{1}{h}\int_K v'(x)\,dx.$$

The interpolant is affine on each element, but it is in general discontinuous across element interfaces. As a result, the global finite element space obtained by assembling these elements is a subspace of $L^2(\Omega)$, but not of $H^1(\Omega)$.

Interpretation. This element illustrates a different philosophy of finite element approximation: instead of interpolating point values, it approximates integral quantities. Such constructions naturally arise in discontinuous Galerkin methods and in mixed formulations, where conservation and element-wise balance properties play a central role. Figure 1.8 shows an example of a finite element interpolation using P_1-discontinuous elements.

These examples illustrate that finite elements can be designed in many different ways, depending on the choice of local approximation spaces and degrees of freedom. While one may invent infinitely many finite elements, two fundamental principles clearly emerge: increasing the a priori smoothness enforced by the element naturally enables the approximation of problems involving higher-order derivatives, whereas increasing the polynomial degree of the local space enhances the accuracy of finite element solutions for sufficiently regular problems.

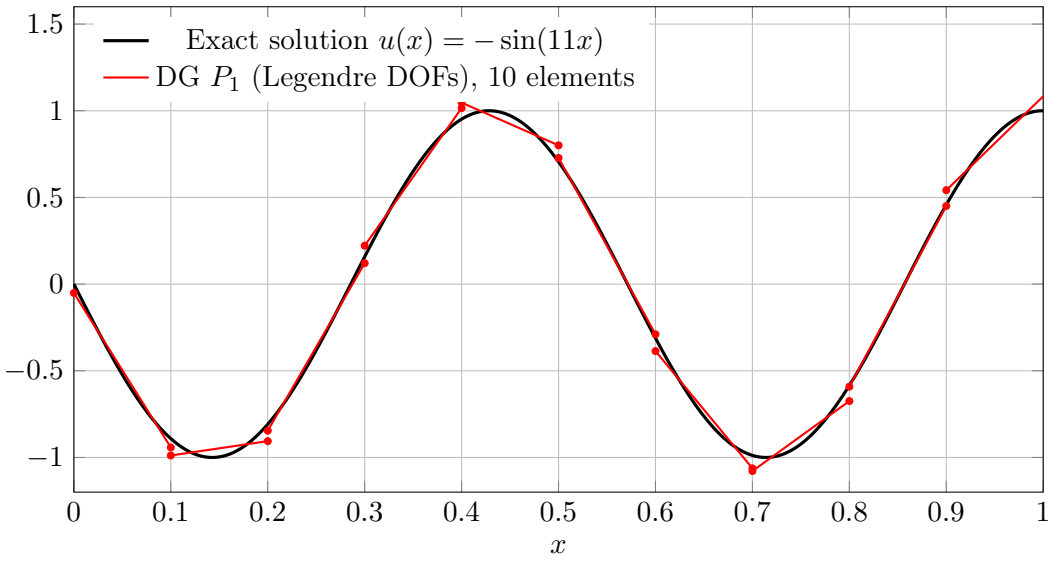

Figure 1.8: Discontinuous P_1 approximation of $u(x) = -\sin(11x)$ on $[0, 1]$ using 10 uniform elements. The approximation is affine on each element and discontinuous at element interfaces. Small red bullets indicate the element-wise endpoint values of the discontinuous interpolant.

1.2.5 The concept of approximation function and support

As we have seen, the finite element method is quite systematic. It starts from a geometric entity called element, then defines a finite-dimensional space of functions over it. The next step is to chose a basis to represent the space with associated degrees of freedom. This is the description for a single finite element. To build a global approximation over a mesh, we need something more. We need to choose the level of regularity that is enforced between one finite element and the next. If continuity is imposed, some degrees of freedom will be shared between elements. If not, degrees of freedom will stay associated to a single element. An approximation function is by definition the function associated to a degree of freedom and its support is by definition the domain over which this function is non-zero. The support of an approximation function can be just one element or a set of elements. Since a degree of freedom is usually associated to node, a support is sometimes called a nodal-patch.

Note that the word support may also be used in a pure geometric way. The support of a vertex refers to the set of elements connected to it (one or two in 1D).

1.3 The one-dimensional Poisson problem

We now turn to the first model problem of this chapter: the Poisson equation with variable diffusion coefficient. This section introduces the variational formulation, the finite element discretization, and the basic a priori error estimate known as Céa's lemma. Let $\Omega = (0, L)$ and let $\kappa \in L^\infty(\Omega)$ satisfy the uniform ellipticity condition

$$0 < \kappa_0 \le \kappa(x) \le \kappa_1 \qquad \text{for all } x \in \Omega, \tag{1.4}$$

for some constants $\kappa_0, \kappa_1 > 0$. Given f, we consider

$$-\frac{d}{dx}\big(\kappa(x)\, u'(x)\big) = f(x) \qquad \text{in } (0, L), \tag{1.5}$$

with homogeneous Dirichlet boundary conditions

$$u(0) = u(L) = 0. \tag{1.6}$$

We introduce the space of admissible functions

$$V := H_0^1(0, L),$$

that is, the Sobolev space of square-integrable functions with square-integrable weak derivatives and vanishing trace at the boundary and define the bilinear and linear forms

$$a(u, v) := \int_0^L \kappa(x)\, u'(x)\, v'(x)\, dx, \qquad \ell(v) := \int_0^L f(x)\, v(x)\, dx. \tag{1.7}$$

The weak formulation reads:

Find $u \in V$ such that

$$a(u, v) = \ell(v) \qquad \forall v \in V. \tag{1.8}$$

Galerkin approximation. Let $V_h \subset V$ be a finite-dimensional subspace (e.g. the P_k finite element space defined just above). The Galerkin finite element approximation reads:

Find $u_h \in V_h$ such that

$$a(u_h, v_h) = \ell(v_h) \qquad \forall v_h \in V_h. \tag{1.9}$$

Galerkin orthogonality. Subtracting (1.9) from (1.8) yields, for all $v_h \in V_h$,

$$a(u - u_h, v_h) = 0. \tag{1.10}$$

This identity is called *Galerkin orthogonality.*

Energy norm, continuity, and coercivity. We define the energy norm associated with $a(\cdot, \cdot)$ by

$$\|v\|_a := \sqrt{a(v, v)}.$$

Using (1.4), we can bound the bilinear form

$$a(v, w) = \int_0^L \kappa(x)\, v'(x)\, w'(x)\, dx \qquad (v, w \in V)$$

as follows. First, since $0 < \kappa(x) \le \kappa_1$ almost everywhere,

$$|a(v, w)| = \left| \int_0^L \kappa(x)\, v'(x)\, w'(x)\, dx \right| \le \int_0^L \kappa(x)\, |v'(x)|\, |w'(x)|\, dx \le \kappa_1 \int_0^L |v'(x)|\, |w'(x)|\, dx.$$

We then apply the Cauchy–Schwarz inequality in $L^2(0, L)$,

$$\int_0^L |v'(x)|\, |w'(x)|\, dx \le \left(\int_0^L |v'(x)|^2\, dx \right)^{1/2} \left(\int_0^L |w'(x)|^2\, dx \right)^{1/2} = \|v'\|_{L^2(0,L)}\, \|w'\|_{L^2(0,L)}.$$

Combining the two estimates yields the continuity bound

$$|a(v, w)| \le \kappa_1 \|v'\|_{L^2(0,L)}\, \|w'\|_{L^2(0,L)}.$$

In particular, $a(\cdot, \cdot)$ is continuous and coercive on V, and $\|\cdot\|_a$ is a norm on V.

1.3.1 Céa's lemma

Lemma 1.1 (Céa)**.** *Assume κ satisfies* (1.4)*. Let $u \in V$ be the solution of* (1.8) *and $u_h \in V_h$ the solution of* (1.9)*. Then*

$$\boxed{\|u - u_h\|_{H^1(0,L)} \leq \sqrt{\frac{\kappa_1}{\kappa_0}} \inf_{v_h \in V_h} \|u - v_h\|_{H^1(0,L)}.} \tag{1.11}$$

Proof. Let $w_h \in V_h$ be arbitrary. By coercivity,

$$\kappa_0 \|u - u_h\|^2_{H^1(0,L)} \leq a(u - u_h, u - u_h) = a(u - u_h,\, u - w_h) + a(u - u_h,\, w_h - u_h).$$

The second term vanishes by Galerkin orthogonality (1.10) since $w_h - u_h \in V_h$. Hence

$$a(u - u_h, u - u_h) = a(u - u_h,\, u - w_h).$$

Using continuity and the definition of the energy norm,

$$\|u - u_h\|_a^2 = a(u - u_h,\, u - w_h) \leq |a(u - u_h,\, u - w_h)| \leq \kappa_1 \|u - u_h\|_{H^1(0,L)} \, \|u - w_h\|_{H^1(0,L)}.$$

A sharper estimate is obtained directly in the energy norm:

$$\|u - u_h\|_a^2 = a(u - u_h,\, u - w_h) \leq \|u - u_h\|_a \, \|u - w_h\|_a.$$

Cancelling $\|u - u_h\|_a$ (which is nonzero unless $u = u_h$) gives

$$\|u - u_h\|_a \leq \|u - w_h\|_a.$$

Finally, we relate $\|\cdot\|_a$ to the H^1-seminorm using (1.4):

$$\|v\|_a^2 = \int_0^L \kappa |v'|^2 \leq \kappa_1 \|v'\|_{L^2}^2, \qquad \|v'\|_{L^2}^2 \leq \frac{1}{\kappa_0} \|v\|_a^2.$$

Combining these bounds yields

$$\|u - u_h\|_{H^1(0,L)} \leq \sqrt{\frac{\kappa_1}{\kappa_0}} \, \|u - w_h\|_{H^1(0,L)}.$$

Since $w_h \in V_h$ was arbitrary, taking the infimum over V_h gives (1.11). □

Interpretation. Céa's lemma states that the Galerkin finite element solution u_h is quasi-optimal: up to the constant κ_1/κ_0, it is as good as the best approximation of u within the discrete space V_h in the energy norm.

1.3.2 From Céa's lemma to convergence: a general approximation result

Céa's lemma shows that the error analysis of the finite element method is entirely governed by approximation properties of the discrete space V_h. More precisely, the finite element error is controlled by the best approximation of the exact solution within V_h measured in the energy norm.

To turn Céa's estimate into a convergence result, one therefore needs an interpolation operator mapping sufficiently smooth functions onto V_h, together with suitable approximation estimates.

The Clément interpolation operator. Let $\mathcal{T}_h$ be a conforming partition of $(0, L)$ into intervals K, and let $V_h \subset H_0^1(0, L)$ be a finite element space of continuous, piecewise polynomials of degree at most p. We denote by $\{\varphi_i\}_{i\in\mathcal{I}_h}$ the set of approximation functions in V_h, associated with the mesh nodes.

For each node x_i, let $\omega_i \subset (0, L)$ denote the *nodal patch*, defined as the union of all elements sharing the node x_i. Note that if it is a internal node, there will be a single element, whereas if it is a vertex node, there will be one or two.

The Clément interpolation operator

$$I_h : H_0^1(0, L) \longrightarrow V_h$$

is defined by

$$I_h u = \sum_{i\in\mathcal{I}_h} \left(\frac{1}{|\omega_i|} \int_{\omega_i} u(x)\, dx \right) \varphi_i(x). \tag{1.12}$$

That is, the coefficient associated with each basis function is given by the average value of u over the corresponding nodal patch

The Clément interpolant is a quasi-interpolation operator: it preserves boundary conditions, is stable in H^1, and provides optimal-order approximation estimates without requiring additional regularity beyond what is naturally assumed for elliptic problems.

Nodal interpolation in practice. In practical finite element computations, the nodal interpolation operator is very often used. This is because, in many engineering applications, the exact solution is smooth enough so that point values are well defined and can be meaningfully evaluated at mesh nodes.

When the solution satisfies additional regularity assumptions, for instance

$$u \in C^0([0, L]) \cap H^{p+1}(0, L),$$

the nodal interpolant is well defined and enjoys optimal approximation properties. In this case, the nodal interpolation error satisfies estimates of the form

$$\|u - \mathcal{I}_h^{\text{nodal}} u\|_{H^1(0,L)} \leq C\, h^p\, \|u\|_{H^{p+1}(0,L)},$$

which are identical to those obtained with quasi-interpolation operators.

As a consequence, for sufficiently smooth solutions, nodal interpolation is perfectly adequate and is often preferred due to its simplicity, ease of implementation, and clear physical interpretation of the degrees of freedom as nodal values.

However, it should be emphasized that this choice relies on additional regularity assumptions on the solution. For this reason, quasi- interpolation operators such as the Clément interpolant are preferred in theoretical analyses, as they remain valid in the minimal regularity framework provided by the variational formulation.

Let $I_h : H_0^1(0, L) \to V_h$ denote a quasi-interpolation operator, such as the Clément interpolant. In Céa's lemma, we choose

$$w_h := I_h u \in V_h.$$

Inserting this choice into (1.11) yields

$$\|u - u_h\|_{H^1(0,L)} \leq \frac{\kappa_1}{\kappa_0}\, \|u - I_h u\|_{H^1(0,L)}. \tag{1.13}$$

Approximation estimate of order p. Assume that the finite element space V_h consists of piecewise polynomials of degree at most p, and that the exact solution satisfies $u \in H^{p+1}(0, L)$. Then the Clément interpolant satisfies the approximation estimate

$$\|u - I_h u\|_{H^1(0,L)} \leq C\, h^p\, \|u\|_{H^{p+1}(0,L)}, \tag{1.14}$$

where h denotes the maximum mesh size and C is a constant independent of h.

Convergence result. Using the equivalence between the energy norm and the H^1-seminorm, which follows from the bounds on κ, we deduce from (1.13)–(1.14) that

$$\boxed{\|u - u_h\|_a \leq C\, h^p\, \|u\|_{H^{p+1}(0,L)}.} \tag{1.15}$$

This estimate proves convergence of order p in the energy norm for finite element spaces of polynomial degree p, provided the exact solution is sufficiently smooth.

Interpretation. This result highlights the two fundamental mechanisms behind finite element convergence. The first one is variational in nature: Céa's lemma ensures quasi-optimality of the Galerkin solution. The second one belongs to approximation theory: higher-degree polynomial spaces yield higher convergence rates when the solution possesses enough regularity.

Final remark on Clément operator The Clément operator does not preserve polynomials except for a linear polynomial.

1.3.3 A complete Python implementation using the `gmsh` API

The problem we want to solve is to find a finite element approximation of the Poisson problem

$$-\frac{d}{dx}\left(\kappa(x)\frac{du}{dx}\right) = f(x)$$

with boundary conditions that will be defined in a separate step. As seen before, we first have to choose a finite element space that is a subset of $H^1([0, L])$ and the natural choice is to choose the continuous finite element space $V_h^{(k)}$ with piecewise polynomial shape functions at order k on each interval K_i. Figure 1.9 is important at that point. We show all shape functions for a 1D uniform mesh made of 4 P^2 elements for a total number of mesh points/degrees of freedom equal to $n = 9$. The figure shows all 9 shape functions corresponding to 5 vertices (mesh points that connect two elements) and internal nodes (mesh points that are inside elements). Every mesh point i has its shape function $\varphi_i(x)$ that has a compact support i.e. it is zero on all elements that do not contain the mesh point. An internal node of an element has a support that is the element itself and a vertex between two elements has those two elements as its support. Another important point is the following – the set of shape functions $(\varphi_1, \ldots, \varphi_9)$ form *a partition of unity*:

$$\sum_{i=1}^{9} \varphi_i(x) = 1 \quad , \quad \forall x \in [0, L].$$

The *partition of unity* property, plays a fundamental role in the definition of finite element shape functions. It guarantees that constant fields are reproduced exactly by the discrete approximation, which is the minimal consistency requirement of any numerical method. As a consequence, nodal values retain a clear physical meaning and rigid-body translations do not generate spurious solutions. Moreover, the ability of the discrete space to represent constant variations is essential for the preservation of global conservation properties, such as mass or energy balance. From a theoretical viewpoint, the partition of unity underlies the consistency assumptions required for stability and optimal convergence of the finite element method, and ensures that local elementwise approximations can be assembled into a globally coherent solution as we will see just after.

We thus approximate the solution $u(x)$ by its finite element approximation

$$u_h(x) = \sum_{i=1}^{n} \varphi_i(x) u_i$$

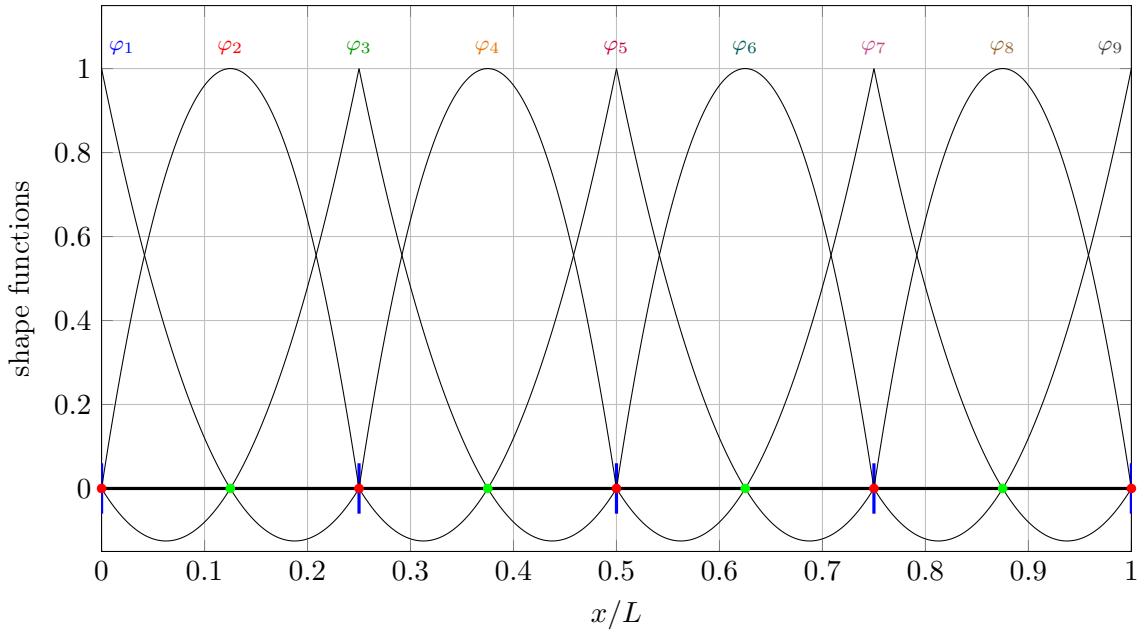

Figure 1.9: A 1D mesh with 4 P_2 elements. The figure shows all 9 shape functions. The function φ_i when i is odd (red points) corresponds to a vertex that is connecting two elements and other shape functions are corresponding to internal nodes (green points). Shape functions have all a compact support that is two elements for vertices and one element for internal nodes.

where $\varphi_i(x)$ is the shape function associated to mesh node i and $u_i = u_h(x_i)$ the ith degree of freedom that correspond to the value of u_h at x_i.

The discrete Dirichlet energy (1.16) writes,

$$\begin{aligned}
\Pi(u_1,\dots,u_n) &= \frac{1}{2}\int_0^L \kappa(x)(u_h'(x))^2 dx - \int_0^L f(x)u_h(x)dx - \cancel{\int_{\Gamma_N} \bar{q}(x)u_h(x)d\Gamma} \\
&= \frac{1}{2}\sum_{i=1}^n\sum_{j=1}^n u_i u_j \underbrace{\int_0^L \kappa(x)\varphi_i'(x)\varphi_j'(x)dx}_{K_{ij}} - \sum_{i=1}^n u_i \underbrace{\int_0^L f\varphi_i(x)dx}_{F_i} \\
&= \frac{1}{2}K_{ij}u_iu_j - u_iF_i \qquad (1.16)
\end{aligned}$$

the last part of the equation assuming Einstein sum convention. We have canceled one term (in red in (1.16)) that correspond to Neumann boundary conditions – we will come later to that. We have to note here that, if u_1 and u_n are unknowns, no Dirichlet boundary condition is applied and the problem we are solving corresponds to $\Gamma = \Gamma_N$ and $\bar{q} = 0$. We will see in the step 5 of the solution process that boundary conditions should be applied *a posteriori*. So for now, we let that problem apart.

We have shown that the unique minimum of the continuous functional $\Pi(u)$ correspond to the solution of the Poisson problem. The discrete version $\Pi(u_1,\dots,u_n)$ is now a function of n variables and we shall find its minimum by simply zeroing its gradient:

$$\frac{d\Pi(u_1,\dots,u_n)}{du_i} = 0 \quad \forall i \quad \rightarrow \quad K_{ij}u_j = F_i.$$

The problem of solving the Poisson problem with finite element consist at the end to solve a linear system of n equations. At that point, we have not yet talked about boundary conditions and we have not yet talked about how to compute the integrals that are needed to compute matrix K_{ij} and vector F_i. We decide here to stop writing equations and move to Python.

Step 1: generating a one-dimensional mesh with polynomial order k. It is indeed possible (and elegant) to write a Python code that is totally agnostic on the degree k of the polynomials. We thus start the finite element implementation of the one-dimensional Poisson problem by generating a mesh of the interval $[0, L]$ using the `gmsh` Python API. The procedure consists of four elementary steps: (i) initialize the `gmsh` environment, (ii) define a straight geometric line of length L along the x-axis, (iii) prescribe two (possibly different) characteristic lengths $\texttt{cl}_1$ and $\texttt{cl}_2$ at the endpoints in order to allow non-uniform meshes and (iv) generate a one-dimensional mesh composed of finite elements of polynomial order k. The polynomial order k controls the degree of the Lagrange finite elements used on each interval: $k = 1$ corresponds to linear elements, $k = 2$ to quadratic elements, and so on.

```
import gmsh

def build_1d_mesh(L=1.0, cl1=0.02, cl2=0.10, k=1):
    gmsh.initialize()
    gmsh.model.add("poisson_1d")

    # Geometry: endpoints with prescribed characteristic lengths
    p0 = gmsh.model.geo.addPoint(0.0, 0.0, 0.0, cl1)
    p1 = gmsh.model.geo.addPoint(L,   0.0, 0.0, cl2)

    # Geometric entity: line [0,L]
    line = gmsh.model.geo.addLine(p0, p1)

    # Synchronize geometry with the mesh module
    gmsh.model.geo.synchronize()

    # Generate a 1D mesh
    gmsh.model.mesh.generate(1)

    # Set the polynomial order of the 1D finite elements
    gmsh.model.mesh.setOrder(k)

    return line
```

Step 2: Gauss quadrature. Numerical integration plays a central role in the finite element method, as most integrals appearing in variational formulations cannot be evaluated analytically on general meshes. In one space dimension, Gauss quadrature provides an efficient and highly accurate integration rule that is particularly well suited to polynomial finite elements.

Principle of Gauss quadrature Let $\hat{K} = [-1, 1]$ be the reference interval. A Gauss quadrature rule with n points approximates the integral of a function f as

$$\int_{-1}^{1} f(\xi)\, d\xi \;\approx\; \sum_{i=1}^{n_g} w_i\, f(\xi_i), \tag{1.17}$$

where $\{\xi_i\}_{i=1}^{n_g} \subset (-1, 1)$ are the Gauss points and $\{w_i\}_{i=1}^{n_g}$ are positive weights. The defining property of Gauss quadrature is its *exactness on polynomials*: the n_g-point Gauss rule integrates exactly all polynomials of degree up to $2n_g - 1$, i.e.

$$\int_{-1}^{1} p(\xi)\, d\xi = \sum_{i=1}^{n_g} w_i\, p(\xi_i) \qquad \forall p \in \mathbb{P}_{2n_g-1}.$$

This optimal degree of exactness explains the efficiency of Gauss quadrature: with a minimal number of integration points, one obtains the highest possible accuracy for polynomial integrands.

Construction of Gauss points and weights. The Gauss points $\{\xi_i\}$ are the roots of the Legendre polynomial $\mathcal{L}_{n_g(\xi)}$ of degree n_g, which is orthogonal on $[-1,1]$ with respect to the L^2 inner product. The corresponding weights are given by

$$w_i = \frac{2}{\big(1-\xi_i^2\big)\,[\mathcal{L}'_{n_g(\xi_i)}]^2}, \qquad i = 1,\dots,n_g. \tag{1.18}$$

In practice, Gauss points and weights are tabulated or provided by numerical libraries like `gmsh`; their explicit computation is rarely required in finite element implementations.

Change of variables to a physical element Let $K = [x_i, x_{i+1}]$ be an element of the physical mesh. Using the affine mapping

$$x(\xi) = \frac{x_{i+1}-x_i}{2}\,\xi + \frac{x_{i+1}+x_i}{2},$$

we have

$$\int_{x_i}^{x_{i+1}} f(x)\,dx = \frac{x_{i+1}-x_i}{2}\int_{-1}^{1} f\big(x(\xi)\big)\,d\xi.$$

The factor $\frac{x_{i+1}-x_i}{2}$ usually called the Jacobian determinant is also provided by `gmsh`. Applying the Gauss rule (1.17) yields

$$\int_{x_i}^{x_{i+1}} f(x)\,dx \;\approx\; \frac{x_{i+1}-x_i}{2}\sum_{j=1}^{n_g} w_j\, f\big(x(\xi_j)\big). \tag{1.19}$$

Choice of quadrature order in the finite element method In the finite element method, the integrands typically involve products of basis functions and their derivatives. For continuous polynomial finite elements of degree k, we have shown that matrix K_{ij} involves integrands that are (without taking into account the fact that $\kappa(x)$ may be variable) polynomials of degree up to $2(k-1)$.

Example: exactness of Gauss quadrature Consider the integral

$$I = \int_{-1}^{1} \big(\xi^4 - \xi^2 + 1\big)\,d\xi.$$

Since the integrand is a polynomial of degree 4, it is integrated exactly by a Gauss rule with $n_g = 3$ points (exact up to degree 5)[1].

Using the three-point Gauss rule, we obtain

$$I = \sum_{i=1}^{3} w_i\big(\xi_i^4 - \xi_i^2 + 1\big),$$

which coincides with the exact analytical value

$$I = \frac{2}{5} - \frac{2}{3} + 2 = \frac{28}{15}.$$

[1]The three-point Gauss–Legendre quadrature rule on the reference interval $[-1,1]$ is given by

$$\int_{-1}^{1} f(\xi)\,d\xi \;\approx\; \sum_{i=1}^{3} w_i\, f(\xi_i),$$

with integration points (abscissa

$$\xi_1 = -\sqrt{\frac{3}{5}}, \qquad \xi_2 = 0, \qquad \xi_3 = \sqrt{\frac{3}{5}},$$

and corresponding weights

$$w_1 = \frac{5}{9}, \qquad w_2 = \frac{8}{9}, \qquad w_3 = \frac{5}{9}.$$

This quadrature rule is exact for all polynomials of degree up to five.

The code. The second step of our Python code consists of preparing all all quantities required for the numerical integration. In practice, `gmsh` is three dimensional and Gauss points are 3D points with only their first coordinate different of zero. Same for shape function gradients that have three coordinates even in 1D. Matrix K_{ij} involves products of derivatives of shape functions φ_i, multiplied by $\kappa(x)$. If we assume that $\kappa(x)$ is not "more complicated" than a piecewise linear polynomial, then, the integrand of K_{ij} is at most at order $2k-1$. Same for F_i, if $f(x)$ is not "more complicated" than a piecewise linear polynomial, then the integrand of F_i is at most of order $k+1$. We choose to use a Gauss quadrature rule that is exact for polynomials up to degree $2k$ which will be good in all cases.

Note here that the reference segment of `gmsh` is indeed $\hat{K} = [-1, 1]$ and everything that follows – integration rules, shape functions and Jacobians – are consistent with this definition (`gmsh` could use another definition of $\hat{K}$ without any change in the Python implementation).

At the same time, we ask `gmsh` to evaluate, at these integration points, the Lagrange shape functions of order k and their gradients with respect to the reference coordinate. It must be noted that the information's provided by function `prepare_quadrature_and_basis` is only related to the reference element and are independent of the mesh size.

```
import numpy as np
import gmsh

def prepare_quadrature_and_basis(elemType, k):
    """
    Returns -- all 1D arrays:
      uvw     : Gauss points on the reference segment (3 x ngp)
      weights : Gauss weights (ngp)
      N       : shape functions Gauss points (ngp x nloc)
      dN_dxi  : reference gradients dN/dxi at Gauss points (3 x ngp x nloc)
    """

    # Gauss rule exact for polynomials of degree 2k (Gmsh rule naming: GaussN)
    rule = f"Gauss{2*k}"

    # Integration points are returned as reference (u,v,w) triplets; in 1D we use u
    uvw, weights = gmsh.model.mesh.getIntegrationPoints(elemType, rule)

    # Evaluate Lagrange basis functions and reference gradients at these points
    _, bf, _  = gmsh.model.mesh.getBasisFunctions(elemType, uvw, "Lagrange")
    _, gbf, _ = gmsh.model.mesh.getBasisFunctions(elemType, uvw, "GradLagrange")

    return elemType, uvw, weights, bf, gbf
```

Step 3: Jacobians Another central concept in the finite element method is the use of mappings between a *reference element* and a *physical element*. The Jacobian of this mapping plays a key role: it accounts for the geometry of the mesh elements and governs how derivatives and integrals are transformed from reference coordinates to physical coordinates.

Jacobian in one space dimension In one dimension, let $\hat{K} = [-1, 1]$ (or $[0, 1]$) be a reference element and let $K = [x_i, x_{i+1}]$ be a physical element of the mesh. The mapping from reference to physical coordinates is given by

$$x = x(\xi) = \sum_{a=1}^{n_{\text{loc}}} x_a \, \varphi_a(\xi),$$

where $\{\varphi_a\}$ are the shape functions on the reference element and $\{x_a\}$ are the physical coordinates of the element nodes. For affine elements (e.g. P_1), this reduces to

$$x(\xi) = \frac{x_{i+1} - x_i}{2}\,\xi + \frac{x_{i+1} + x_i}{2}.$$

The Jacobian of the transformation is the scalar

$$J(\xi) := \frac{dx}{d\xi}.$$

It measures the local stretching of the element. Using the chain rule, the derivative of a finite element function u_h satisfies

$$\frac{du_h}{dx} = \frac{du_h}{d\xi}\,\frac{d\xi}{dx} = \frac{1}{J(\xi)}\,\frac{du_h}{d\xi}.$$

Similarly, integrals transform according to

$$\int_K f(x)\,dx = \int_{\hat{K}} f\big(x(\xi)\big)\,|J(\xi)|\,d\xi.$$

Thus, in one dimension, the Jacobian simultaneously controls the evaluation of derivatives and the correct scaling of integrals.

Jacobian matrix in three space dimensions. This Chapter deals with 1D elements. Yet everything that is said could be extended in 3D – shape functions, partition of unity, quadrature rules. For Jacobians , the situation is very analogous but in 3D, the Jacobian becomes a matrix. Let $\hat{K} \subset \mathbb{R}^3$ be a reference element (e.g. a tetrahedron or a hexahedron) with coordinates $\boldsymbol{\xi} = (\xi_1, \xi_2, \xi_3)$, and let $K \subset \mathbb{R}^3$ be the corresponding physical element with coordinates $\mathbf{x} = (x_1, x_2, x_3)$. The isoparametric mapping reads

$$\mathbf{x}(\boldsymbol{\xi}) = \sum_{a=1}^{n_{\mathrm{loc}}} \mathbf{x}_a\,\varphi_a(\boldsymbol{\xi}),$$

where $\mathbf{x}_a \in \mathbb{R}^3$ are the nodal coordinates.

The Jacobian matrix of the transformation is defined as

$$\mathbf{J}(\boldsymbol{\xi}) = \frac{\partial \mathbf{x}}{\partial \boldsymbol{\xi}} = \begin{pmatrix} \dfrac{\partial x_1}{\partial \xi_1} & \dfrac{\partial x_1}{\partial \xi_2} & \dfrac{\partial x_1}{\partial \xi_3} \\ \dfrac{\partial x_2}{\partial \xi_1} & \dfrac{\partial x_2}{\partial \xi_2} & \dfrac{\partial x_2}{\partial \xi_3} \\ \dfrac{\partial x_3}{\partial \xi_1} & \dfrac{\partial x_3}{\partial \xi_2} & \dfrac{\partial x_3}{\partial \xi_3} \end{pmatrix}.$$

Its determinant $\det \mathbf{J}$ measures the local change of volume between the reference and physical elements.

Transformation of gradients Let u_h be a finite element function. Its gradient with respect to the physical coordinates is obtained from the gradient in reference coordinates by the chain rule:

$$\nabla_{\mathbf{x}} u_h = \mathbf{J}^{-T}\,\nabla_{\boldsymbol{\xi}} u_h.$$

This formula is fundamental in finite element implementations: basis function derivatives are typically evaluated on the reference element, and the inverse transpose of the Jacobian maps them to physical space.

Similarly, volume integrals transform as

$$\int_K f(\mathbf{x})\,d\mathbf{x} = \int_{\hat{K}} f\big(\mathbf{x}(\boldsymbol{\xi})\big)\,|\det \mathbf{J}(\boldsymbol{\xi})|\,d\boldsymbol{\xi}.$$

The code. The third step of our Python code consists in computing the jacobians at every integration point of every element of the mesh. It requires to query mesh nodes positions as well as the connectivity of the mesh. Jacobians are global informations so we need at first a mesh and then an integration rule. The `gmsh` API provides a direct access to the Jacobians (3×3 matrices), their determinants (reals) and the spatial coordinates of the integration points on every of a given `elementType` that will be very useful for computing $\kappa(x)$ et $f(x)$. Gmsh works in 3D so jacobians are 3×3 matrices. The beauty here is that i) every jacobian is invertible, ii) you cal solve 1D the problem along y or z or on a curve and most importantly iii) the code we write will be the same for triangles (2D) or tetrahedra (3D).

```
"""
assume ngp gauss points, ne elements and nn mesh points per element
gmsh only returns 1D arrays
jacobians is an array of size (ne x ngp x 3 x 3)
dets      is an array of size (ne x ngp)
xyzcoord  is an array of size (ne x ngp x 3)
"""
jacobians, dets, xyzcoord = gmsh.model.mesh.getJacobians(elemType, uvw)
```

Step 4: Assembly of the global system. We now *assemble* the linear system associated with the one-dimensional Poisson problem. The goal of this *assembly* step is to build the global Laplace matrix $K \in \mathbb{R}^{n \times n}$ and the global right-hand side vector $F \in \mathbb{R}^n$.

The assembly process can be understood from varous viewpoints. The one we choose is of physical nature. The Dirichlet energy Π is indeed a true energy, defined as an integral over the whole domain $[0, L]$. As such, it is an extensive quantity: the energy associated with the entire mesh is equal to the sum of the energies stored in each individual element. This observation naturally leads to an *element-by-element* construction of the Laplace matrix K.

If the energy of a single element were to depend on all n nodal values of the mesh, such a procedure would be counterproductive, and one would rather attempt to organize the computation node by node. This is not the case in the finite element method. The only nonzero contributions to the energy of a given element arise from the degrees of freedom whose associated shape functions have a nonzero support on that element, namely the nodes of the element itself. This locality property is a direct consequence of the compact support of finite element shape functions. Formally, if $\mathcal{N}_e$ is the set of mesh nodes of element e, we have

$$\begin{aligned} \Pi(u_1, \dots, u_n) &= \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} u_i u_j \underbrace{\int_0^L \kappa(x) \varphi_i'(x) \varphi_j'(x) dx}_{K_{ij}} - \sum_{i=1}^{n} u_i \underbrace{\int_0^L f(x) \varphi_i(x) dx}_{F_i} \qquad (1.20) \\ &= \frac{1}{2} \sum_{e=1}^{n_e} \left[\sum_{a \in \mathcal{N}_e} \sum_{b \in \mathcal{N}_e} u_a u_b \underbrace{\int_e \kappa(x) \varphi_a'(x) \varphi_b'(x) dx}_{K_{ab}^e} - \sum_{a \in \mathcal{N}_e} \underbrace{\int_e f(x) \varphi_a(x) dx}_{F_a^e} \right]. \end{aligned}$$

In (1.21), matrix $K^e \in \mathbb{R}^{n_{\text{loc}}} \times n_{\text{loc}}$ and can be computed as

$$K_{ab}^e = \int_{K^e} \kappa(x) \, \frac{d\varphi_a}{dx}(x) \, \frac{d\varphi_b}{dx}(x) \, dx, \qquad a, b = 1, \dots, n_{\text{loc}},$$

where $\{\varphi_a\}_{a=1}^{n_{\text{loc}}}$ denotes the local shape functions associated with element e. The global stiffness matrix K is then obtained by assembling these local contributions into the appropriate positions according to the mesh connectivity. This procedure mirrors exactly the additivity of the Dirichlet energy and provides a clear physical interpretation of the algebraic assembly process.

Elementwise contributions. Let $K = [x_e^-, x_e^+]$ be an element of the mesh. Denoting by $\{\phi_a\}_{a=1}^{n_{\text{loc}}}$ the local Lagrange basis of order k on the reference segment $\hat{K}$, the element stiffness matrix $\mathbf{K}^e \in \mathbb{R}^{n_{\text{loc}} \times n_{\text{loc}}}$ and load vector $\mathbf{F}^e \in \mathbb{R}^{n_{\text{loc}}}$ are defined by

$$K_{ab}^e = \int_K \kappa(x)\, \phi_a'(x)\, \phi_b'(x)\, dx, \qquad F_a^e = \int_K f(x)\, \phi_a(x)\, dx.$$

These integrals are evaluated by Gauss quadrature. For a rule of order $2k$, we obtain

$$K_{ab}^e \approx \sum_{g=1}^{n_g} w_g\, \kappa(x_g)\, \phi_a'(x_g)\, \phi_b'(x_g)\, |J_g|, \qquad F_a^e \approx \sum_{g=1}^{n_g} w_g\, f(x_g)\, \phi_a(x_g)\, |J_g|,$$

where w_g are the Gauss weights on the reference element, x_g are the physical coordinates of the quadrature points, and $|J_g|$ is the absolute value of the Jacobian determinant of the map from the reference element to the physical element at the Gauss point.

Derivative transformation. The shape function gradients returned by `gmsh` are reference gradients, i.e. derivatives with respect to the reference coordinate ξ. In one dimension, the chain rule gives

$$\frac{d\phi_a}{dx}(x_g) = \frac{d\phi_a}{d\xi}(\xi_g) \left(\frac{dx}{d\xi}(\xi_g)\right)^{-1}.$$

In the implementation, `gmsh.model.mesh.getJacobians` is used to obtain $\frac{dx}{d\xi}$ (via the Jacobian matrix) and $|J_g| = |\det J|$ at the Gauss points for each physical element.

Scatter-add to the global system. Let $\{i_a\}_{a=1}^{n_{\text{loc}}}$ be the global indices of the degrees of freedom of element e (the connectivity array). The global assembly is performed by the standard scatter-add operation

$$K_{i_a i_b} \mathrel{+}= K_{ab}^e, \qquad F_{i_a} \mathrel{+}= F_a^e, \qquad a, b = 1, \ldots, n_{\text{loc}},$$

looping over all elements and all Gauss points. In the code, this is implemented with explicit nested loops over elements, quadrature points, and local indices. The global matrix is stored in a sparse format to reflect the locality of the finite element basis.

Assemblage analytique en 1D : 4 éléments P_1, $h = \frac{1}{4}$, $\kappa = 1$ On considère $\Omega = [0, 1]$ et un maillage uniforme de 4 éléments

$$e_1 = [x_1, x_2], \quad e_2 = [x_2, x_3], \quad e_3 = [x_3, x_4], \quad e_4 = [x_4, x_5], \qquad x_i = \frac{i-1}{4}, \quad h = x_{i+1} - x_i = \frac{1}{4}.$$

Les noeuds globaux sont numérotés de 1 à 5.

Sur un élément $[x_i, x_{i+1}]$, les fonctions de forme P_1 sont

$$N_1(x) = \frac{x_{i+1} - x}{h}, \qquad N_2(x) = \frac{x - x_i}{h},$$

d'où

$$N_1'(x) = -\frac{1}{h}, \qquad N_2'(x) = \frac{1}{h}.$$

Pour $\kappa = 1$, la matrice de raideur locale vaut

$$K^e = \int_e \begin{bmatrix} N_1' \\ N_2' \end{bmatrix} \begin{bmatrix} N_1' & N_2' \end{bmatrix} dx = \int_{x_i}^{x_{i+1}} \frac{1}{h^2} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} dx = \frac{1}{h} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

et est identique pour les 4 éléments. Comme $h = \frac{1}{4}$, on obtient

$$\boxed{K^e = 4\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} = \begin{bmatrix} 4 & -4 \\ -4 & 4 \end{bmatrix}.}$$

To assemble the global matrix K that is 5×5, we start with an empty matrix

$$K^{(0)} = \mathbf{0}_{5\times 5}.$$

Each element e_i couples nodes $(i, i+1)$. We add K^e in the right rows and columns.

Element e_1 : $(1, 2)$. We add K^e at indices $(1, 2) \times (1, 2)$:

$$K^{(1)} = \begin{bmatrix} 4 & -4 & 0 & 0 & 0 \\ -4 & 4 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

Element e_2 : $(2, 3)$. We add K^e at indices $(2, 3) \times (2, 3)$:

$$K^{(2)} = \begin{bmatrix} 4 & -4 & 0 & 0 & 0 \\ -4 & 8 & -4 & 0 & 0 \\ 0 & -4 & 4 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

Element e_3 : $(3, 4)$. We add K^e at indices $(3, 4) \times (3, 4)$:

$$K^{(3)} = \begin{bmatrix} 4 & -4 & 0 & 0 & 0 \\ -4 & 8 & -4 & 0 & 0 \\ 0 & -4 & 8 & -4 & 0 \\ 0 & 0 & -4 & 4 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

Element e_4 : $(4, 5)$. We add K^e at indices $(4, 5) \times (4, 5)$:

$$\boxed{K^{(4)} = K = \begin{bmatrix} 4 & -4 & 0 & 0 & 0 \\ -4 & 8 & -4 & 0 & 0 \\ 0 & -4 & 8 & -4 & 0 \\ 0 & 0 & -4 & 8 & -4 \\ 0 & 0 & 0 & -4 & 4 \end{bmatrix}.} \tag{1.21}$$

This matrix is exactly equivalent to a standard finite difference scheme which is perfectly normal. Pr. Gilbert Strang on his famous book on finite elements [2] started by this quote:

> "The finite element method is a systematic procedure for constructing finite difference equations."

The code. In the code, we have not looked the problem as being 1D, it is agnostic both to polynomial order k and dimension d. For `gmsh`, a jacobian is a 3×3 matrix and shape functions have gradients that are 3-dimensional vectors, whatever the dimension of the element. This is indeed very interesting because the following code will be usable in 2D (even on non planar surfaces) and in 3D. Another advantage of that implementation is that, even though we are not using the vectorization capabilities of NumPy, the code can be easily accelerated using *just in time (JIT) compilation*. In standard Python execution, instructions are interpreted one by one, which makes tight loops and low-level numerical kernels relatively slow. Just-in-time (JIT) compilation alleviates this limitation by translating selected Python functions into optimized machine code *at runtime*. The compilation is triggered the first time the function is called, once the types of its arguments are known; subsequent calls then execute the compiled version directly.

Although JIT compilation can dramatically accelerate numerical kernels, it does not support the full Python language. In particular, libraries such as `Numba` only accept a restricted subset of Python and NumPy, and do not handle high-level constructs such as dictionaries, dynamic lists, classes, or calls to external libraries.

As a consequence, an effective programming pattern consists in *separating data preparation from computation*. All high-level operations (mesh generation, connectivity, quadrature rules, evaluation of basis functions, user-defined coefficients) are first performed in standard Python, and the resulting data are stored in plain NumPy arrays. The JIT-compiled functions are then limited to the innermost computational kernels, typically composed of nested loops and basic arithmetic operations acting exclusively on these precomputed arrays. This separation ensures both correctness and performance, and mirrors the classical structure of finite element codes written in compiled languages. This is exactly what we have done here.

```
import numpy as np
from scipy.sparse import lil_matrix

def assemble_K_F(elemType, elemTags, conn, jac, det,
                 xphys, uvw, w, N, gN, kappa_fun, rhs_fun):


    ne = len(elemTags) # number of mesh elements
    nn = int(np.max(conn)) # number of mesh nodes
    nloc = int(len(conn)/ne) # number of mesh nodes per element
    ngp = len(w) # number of integration points per element

    # - Reshape data for making things easy
    det = np.array(det, dtype=np.float64).reshape(ne, ngp)
    xphys = np.array(xphys, dtype=np.float64).reshape(ne, ngp, 3)
    jac = np.array(jac, dtype=np.float64).reshape(ne, ngp, 3, 3)
    conn = np.array(conn, dtype=np.int64).reshape(ne, nloc)
    N = np.array(N, dtype=np.float64).reshape(ngp, nloc)
    gN = np.array(gN, dtype=np.float64).reshape(ngp, nloc,3)

    # Allocate global matrix/vector
    K = lil_matrix((nn, nn), dtype=np.float64)
    F = np.zeros(nn, dtype=np.float64)

    # Assembly loops (no vectorization)
    for e in range(ne):
        nodes = conn[e, :] - 1  # gmsh number nodes from 1 to n

        for g in range(ngp):
            xg   = xphys[e, g]
            wg   = w[g]
            detg = det[e, g]
```

```
            jacg = jac[e, g]
            invjacg = np.linalg.inv(jacg)
            kappa_g = float(kappa_fun(xg))
            f_g = float(rhs_fun(xg))
            for a in range(nloc):
                Ia = int(nodes[a])
                F[Ia] += wg * f_g * N[g, a] * detg
                gradFa = invjacg@gN[g, a]
                # Stiffness
                for b in range(nloc):
                    Ib = int(nodes[b])
                    gradFb = invjacg@gN[g, b]
                    K[Ia, Ib] += wg * kappa_g * np.dot(gradFa,gradFb) * detg

    return K, F
```

Step 5: Dirichlet boundary conditions. Looking at matrix (1.21), we it is easy to prove that it is singular – which is perfectly normal. In the general case of the Poisson problem, it is well known that the solution is unique only if the value of the solution is prescribed at least at one point of the domain.

Recall that Dirichlet boundary conditions must be enforced *a priori* by fixing the degrees of freedom associated with the prescribed boundary values. If, as we have done so far, we do nothing—that is, if we leave all degrees of freedom free—then the discrete problem behaves as if the whole boundary of the domain, namely the two endpoints $x = 0$ and $x = L$ of the interval, were subject to homogeneous Neumann conditions, $\bar{q} = 0$, on $\partial\Omega$.

We thus need to apply at least one Dirichlet boundary condition. In the finite element framework, this is classically achieved by providing a list of mesh nodes that define the Dirichlet boundary Γ_D, together with a function $d(x)$ that specifies the values to be imposed on this boundary.

Let $K\mathbf{u} = F$ be the linear system obtained after assembly, where $\mathbf{u} \in \mathbb{R}^n$ collects the nodal degrees of freedom. We assume that Dirichlet boundary conditions are prescribed on a set of indices

$$I_D \subset \{1, \dots, n\},$$

and that the corresponding nodal values are given by

$$u_i = d_i \qquad \text{for all } i \in I_D,$$

where $d_i = d(x_i)$ are known. We denote by

$$I_F = \{1, \dots, n\} \setminus I_D$$

the complementary set of $f$ree degrees of freedom.

We reorder the unknowns accordingly and decompose the system into block form:

$$\begin{pmatrix} K_{FF} & K_{FD} \\ K_{DF} & K_{DD} \end{pmatrix} \begin{pmatrix} \mathbf{u}_F \\ \mathbf{u}_D \end{pmatrix} = \begin{pmatrix} \mathbf{f}_F \\ \mathbf{f}_D \end{pmatrix},$$

where the subscripts F and D refer to free and Dirichlet degrees of freedom, respectively. Since $\mathbf{U}_D = \mathbf{d}$ is known, the second block equation is discarded and the first one yields the reduced system

$$K_{FF}\,\mathbf{u}_F = \mathbf{f}_F - K_{FD}\,\mathbf{d}.$$

This reduced system involves only the unknown free degrees of freedom and has a unique solution provided that at least one Dirichlet condition has been imposed. Once $\mathbf{u}_F$ has been computed, the full solution vector $\mathbf{U}$ is recovered by inserting the prescribed values $\mathbf{u}_D = \mathbf{d}$.

Interpretation. From a variational viewpoint, this procedure amounts to restricting the finite element space to functions that satisfy the Dirichlet boundary conditions exactly. From an algebraic viewpoint, it corresponds to eliminating the constrained degrees of freedom and modifying the right- hand side to account for their contribution. This approach preserves the symmetry and positive definiteness of the stiffness matrix and is the standard way of imposing Dirichlet conditions in finite element implementations.

There exist other, less elegant, ways of enforcing Dirichlet boundary conditions. Some of them destroy the symmetry and/or the positive definiteness of the stiffness matrix $\mathbf{K}$. Other approaches introduce Lagrange multipliers and enforce Dirichlet constraints *from the outside*. Lagrange multipliers can be an attractive option, since their values often have a direct physical interpretation. For instance, in solid mechanics, if a displacement is prescribed in a given direction, then the associated Lagrange multiplier represents the reaction force in that direction.

The Code. An additional advantage of the elimination strategy presented above is its excellent compatibility with the Python language: implementing the reduced system is straightforward once the sets of free and Dirichlet degrees of freedom have been identified.

```
def apply_dirichlet_by_reduction(K, F, dirichlet_dofs, dirichlet_values):

    n = len(F)

    # Build free dofs = complement of dirichlet dofs
    mask = np.ones(n, dtype=bool)
    mask[dirichlet_dofs] = False
    free_dofs = np.nonzero(mask)[0]

    # Reduced blocks
    K_FF = K[free_dofs, :][:, free_dofs]
    K_FD = K[free_dofs, :][:, dirichlet_dofs]

    # Reduced RHS: F_F - K_FD d
    F_F = F[free_dofs]
    F_red = F_F - K_FD.dot(dirichlet_values)

    # Build a full vector with prescribed values (free part to be filled after solve)
    U_full = np.zeros(n, dtype=float)
    U_full[dirichlet_dofs] = dirichlet_values

    return K_FF, F_red, free_dofs, U_full
```

Step 6: The finite element solution can be exact at mesh nodes. We can then solve the full Dirichlet problem using the following code that actually solves the problem $\nabla^2 u = 1$ on a domain of size L with Dirichlet boundary conditions $u(0) = 0$ and $u(L) = 1$. (we know that the two nodes the correspond to those points are the two first nodes of the mesh, in what follows we will see how to give "names" to boundaries and assign given boundary conditions on given names).

```
import gmsh
import numpy as np
import argparse
from scipy.sparse import lil_matrix
from scipy.sparse import csr_matrix
from scipy.sparse.linalg import spsolve

if __name__ == "__main__":
```

```
parser.add_argument("-order", type=int, default=1,help="Polynomial order")
parser.add_argument("-cl1", type=float, default=0.2, help="Mesh size at x=0")
parser.add_argument("-cl2", type=float, default=0.2, help="Mesh size at x=L")
parser.add_argument("-L",   type=float, default=1.0, help="Length of domain")
args = parser.parse_args()

# create the mesh
line_tag = build_1d_mesh(L=args.L, cl1=args.cl1, cl2=args.cl2, k=args.order)

# compute quadrature points and shape functions and their gradients at those points
elemType, nloc, xi, w, N, dN_dxi = prepare_quadrature_and_basis(args.order)

# compute jacobians, determinants and Gauss points positions
jacobians, dets, coord = gmsh.model.mesh.getJacobians(elemType, xi)

# defined user functions -- we solve \nabla^2 u = 1
def kappa(x): return 1.0
def f(x):     return 1.0

# Assemble the full laplacian matrix and the full right hand side
K_lil, F = assemble_K_F(elemType, elemTags, elemNodeTags, jacobians,
                        dets, coord, xi, w, N, gN, kappa, f)

# create the reduced system -- homogeneous Dirichlet at both sides
dirichlet_dofs   = [0,1]
dirichlet_values = [0.0,1.0]
K = K_lil.tocsr()
K_red, F_red, free_dofs, U_full = apply_dirichlet_by_reduction(K, F,
                        dirichlet_dofs, dirichlet_values)

# solve reduced system
U_free = spsolve(K_red, F_red)

# reconstruct the full solution
U_full[free_dofs] = U_free
U_full[dirichlet_dofs] = dirichlet_values
```

The exact solution of the problem is quite simple:

$$u(x) = \frac{1}{2L} x(-L^2 + Lx + 2).$$

We restart from the same global stiffness matrix (4 uniform P_1 elements, $h = \frac{1}{4}$, nodes numbered $1, \ldots, 5$):

$$K = \begin{bmatrix} 4 & -4 & 0 & 0 & 0 \\ -4 & 8 & -4 & 0 & 0 \\ 0 & -4 & 8 & -4 & 0 \\ 0 & 0 & -4 & 8 & -4 \\ 0 & 0 & 0 & -4 & 4 \end{bmatrix}.$$

For P_1 elements and constant right-hand side $f = 1$, the consistent element load is $\mathbf{f}_e =$

$\frac{h}{2}[1,1]^T$. Assembling the four elements yields

$$\mathbf{f} = \begin{bmatrix} \frac{h}{2} \\ h \\ h \\ h \\ \frac{h}{2} \end{bmatrix} = \begin{bmatrix} \frac{1}{8} \\ \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{8} \end{bmatrix}.$$

Let $I_D = \{1, 5\}$ be the Dirichlet indices and $I_F = \{2, 3, 4\}$ the free indices. We split the unknown vector into free and prescribed components,

$$\mathbf{U} = \begin{bmatrix} \mathbf{u}_F \\ \mathbf{u}_D \end{bmatrix}, \qquad \mathbf{u}_F = \begin{bmatrix} u_2 \\ u_3 \\ u_4 \end{bmatrix}, \qquad \mathbf{u}_D = \begin{bmatrix} u_1 \\ u_5 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Accordingly, the global system $\mathbf{KU} = \mathbf{F}$ can be written in block form as

$$\begin{bmatrix} K_{FF} & K_{FD} \\ K_{DF} & K_{DD} \end{bmatrix} \begin{bmatrix} \mathbf{u}_F \\ \mathbf{u}_D \end{bmatrix} = \begin{bmatrix} \mathbf{f}_F \\ \mathbf{f}_D \end{bmatrix}.$$

Extracting the relevant rows/columns gives

$$K_{FF} = \begin{bmatrix} 8 & -4 & 0 \\ -4 & 8 & -4 \\ 0 & -4 & 8 \end{bmatrix}, \qquad K_{FD} = \begin{bmatrix} -4 & 0 \\ 0 & 0 \\ 0 & -4 \end{bmatrix},$$

and

$$\mathbf{f}_F = \begin{bmatrix} \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \end{bmatrix}, \qquad \mathbf{f}_D = \begin{bmatrix} \frac{1}{8} \\ \frac{1}{8} \end{bmatrix}.$$

Since $\mathbf{U}_D$ is known, the first block row yields the reduced system

$$\boxed{K_{FF}\,\mathbf{u}_F = \mathbf{f}_F - K_{FD}\,\mathbf{u}_D.}$$

Here,

$$K_{FD}\mathbf{u}_D = \begin{bmatrix} -4 & 0 \\ 0 & 0 \\ 0 & -4 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ -4 \end{bmatrix},$$

so the modified right-hand side is

$$\mathbf{f}_F - K_{FD}\mathbf{u}_D = \begin{bmatrix} \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ -4 \end{bmatrix} = \begin{bmatrix} \frac{1}{4} \\ \frac{1}{4} \\ \frac{17}{4} \end{bmatrix}.$$

Therefore, we solve

$$\begin{bmatrix} 8 & -4 & 0 \\ -4 & 8 & -4 \\ 0 & -4 & 8 \end{bmatrix} \begin{bmatrix} u_2 \\ u_3 \\ u_4 \end{bmatrix} = \begin{bmatrix} \frac{1}{4} \\ \frac{1}{4} \\ \frac{17}{4} \end{bmatrix},$$

which gives

$$u_2 = \frac{11}{32}, \qquad u_3 = \frac{5}{8}, \qquad u_4 = \frac{27}{32}.$$

Finally, inserting $u_1 = 0$ and $u_5 = 1$, we obtain

$$\boxed{\mathbf{u} = \begin{bmatrix} 0 \\ \frac{11}{32} \\ \frac{5}{8} \\ \frac{27}{32} \\ 1 \end{bmatrix}}.$$

Figure 1.10 shows both the exact solution of the problem and the finite element solution. The finite element solution is exact at the mesh nodes, and this is not a coincidence. In fact, under certain conditions—which we now state and justify—the finite element solution coincides exactly with the exact solution at the nodal points of the mesh.

Conditions for nodal exactness – the 1D Poisson case. Consider the one-dimensional Poisson problem

$$-u''(x) = f(x) \quad \text{in } (0, L), \qquad u(0) = \alpha, \quad u(L) = \beta,$$

and let u_h be the finite element solution obtained with continuous P_1 elements on a *uniform mesh* of size h. The finite element solution is exact at the mesh nodes provided that:

- the diffusion coefficient is constant, $\kappa(x) \equiv \kappa$;
- the mesh is uniform;
- the right-hand side satisfies $f \in C^0([0, L])$;
- the load vector is computed exactly (or using a quadrature rule exact for polynomials of degree at least 1).

Under these assumptions,

$$u_h(x_i) = u(x_i) \qquad \text{for all mesh nodes } x_i.$$

Proof. On a uniform mesh with P_1 elements, the finite element method leads to the discrete equations

$$\frac{2}{h}u_i - \frac{1}{h}u_{i-1} - \frac{1}{h}u_{i+1} = \int_{x_{i-1}}^{x_{i+1}} f(x)\,\varphi_i(x)\,dx, \qquad i = 1, \ldots, n-1,$$

where $\{\varphi_i\}$ are the P_1 shape functions. The left-hand side coincides with the classical centered finite difference operator.

Let u denote the exact solution. A Taylor expansion of u about x_i yields

$$u(x_{i\pm1}) = u(x_i) \pm h u'(x_i) + \frac{h^2}{2}u''(x_i) \pm \frac{h^3}{6}u^{(3)}(x_i) + \mathcal{O}(h^4).$$

Combining these expansions gives

$$\frac{2}{h}u(x_i) - \frac{1}{h}u(x_{i-1}) - \frac{1}{h}u(x_{i+1}) = -h u''(x_i) + \mathcal{O}(h^3).$$

Using the differential equation $-u''(x_i) = f(x_i)$, we obtain

$$\frac{2}{h}u(x_i) - \frac{1}{h}u(x_{i-1}) - \frac{1}{h}u(x_{i+1}) = h f(x_i) + \mathcal{O}(h^3).$$

On the other hand, the right-hand side satisfies

$$\int_{x_{i-1}}^{x_{i+1}} f(x)\,\varphi_i(x)\,dx = h f(x_i) + \mathcal{O}(h^3),$$

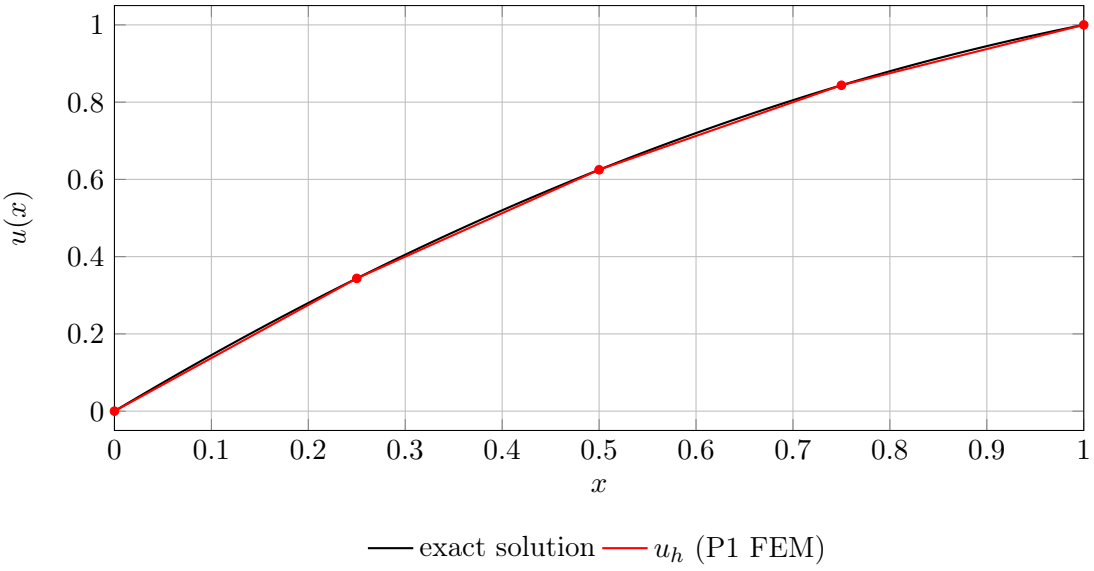

Figure 1.10: Exact solution vs. finite element solution for $u'' = 1$, $u(0) = 0$, $u(1) = 1$.

and this equality is exact if f is constant or affine on $[x_{i-1}, x_{i+1}]$.

Hence, the exact solution u satisfies the same discrete equations as the finite element solution at the mesh nodes. Since the discrete system has a unique solution once the Dirichlet boundary conditions are imposed, it follows that

$$u_i = u(x_i) \qquad \text{for all } i.$$

Remark. This nodal exactness is a special property of the one-dimensional Poisson problem with constant coefficients on a uniform mesh. It generally disappears for nonuniform meshes, variable coefficients, or in higher dimensions.

Nodal exactness for higher-order elements. The nodal exactness property extends to higher-order finite elements in a very specific sense. Consider the one-dimensional Poisson problem

$$-(\kappa(x)u'(x))' = f(x) \quad \text{in } (0, L),$$

discretized with continuous P_k Lagrange finite elements on a *uniform mesh*, using exact integration.

If the diffusion coefficient is constant,

$$\kappa(x) \equiv \kappa,$$

and the right-hand side f is such that the exact solution u is a polynomial of degree at most $k + 1$ (equivalently, f is a polynomial of degree at most $k - 1$), then the finite element solution satisfies

$$u_h(x_i) = u(x_i) \qquad \text{for all mesh nodes } x_i.$$

In particular, the nodal values of the finite element solution coincide exactly with those of the exact solution.

If κ is variable or if the mesh is nonuniform, this nodal exactness property is generally lost, even when higher-order elements are employed. In that case, the finite element solution remains accurate in an optimal approximation sense, but it no longer interpolates the exact solution at the mesh nodes.

Step 7. Neumann Boundary Conditions Up to now, we only considered Dirichlet boundary conditions that are imposed *a priori* or strongly. In Equation (1.16), we have deliberately omitted the boundary term on Γ_N, since this part of the boundary is empty in the case of a purely Dirichlet problem. Let us now rewrite the Dirichlet energy in its full form.

Assume that a Neumann boundary condition $\kappa \frac{du}{dx} \cdot n = \bar{q}$ is imposed on $x = 0$, the full discrete Dirichlet energy (**??**) writes,

$$\begin{aligned}
\Pi(u_1, \dots, u_n) &= \frac{1}{2} \int_0^L \kappa(x)(u_h'(x))^2 dx - \int_0^L f(x) u_h(x) dx - \bar{q} u_h(0) \\
&= \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n u_i u_j \underbrace{\int_0^L \kappa(x) \varphi_i'(x) \varphi_j'(x) dx}_{K_{ij}} - \sum_{i=1}^n u_i \underbrace{\int_0^L f \varphi_i(x) dx}_{F_i} - \bar{q} u_0 \phi_0(0) \\
&= \frac{1}{2} K_{ij} u_i u_j - u_i F_i - u_0 \bar{q}. \qquad (1.22)
\end{aligned}$$

We consider the 1D Poisson problem

$$-u''(x) = 1 \quad \text{in } (0,1), \qquad \partial_n u(0) = 1, \qquad u(1) = 1,$$

with $\kappa = 1$. In 1D, the outward normal at $x = 0$ is $n = -1$, so $\partial_n u(0) = u'(0)\, n = -u'(0) = 1$, i.e.

$$u'(0) = -1.$$

Exact (continuous) solution

Integrating twice gives $u'' = -1$, hence

$$u'(x) = -x + C_1, \qquad u(x) = -\frac{x^2}{2} + C_1 x + C_2.$$

Using $u'(0) = -1$ yields $C_1 = -1$. Then $u(1) = 1$ gives $-\frac{1}{2} - 1 + C_2 = 1 \Rightarrow C_2 = \frac{5}{2}$. Therefore

$$\boxed{u(x) = -\frac{x^2}{2} - x + \frac{5}{2}.}$$

Discrete system on the mesh (4 uniform P_1 elements)

We reuse the global stiffness matrix (nodes numbered $1, \dots, 5$, $h = 1/4$)

$$K = \begin{bmatrix} 4 & -4 & 0 & 0 & 0 \\ -4 & 8 & -4 & 0 & 0 \\ 0 & -4 & 8 & -4 & 0 \\ 0 & 0 & -4 & 8 & -4 \\ 0 & 0 & 0 & -4 & 4 \end{bmatrix}.$$

Load vector. For $f = 1$, the assembled (consistent) body-force load vector is

$$\mathbf{f}^{\text{vol}} = \begin{bmatrix} \frac{1}{8} \\ \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{8} \end{bmatrix}.$$

The Neumann condition contributes a boundary term in the weak form, $\int_{\Gamma_N} \bar{q}\, v\, ds$. Here $\Gamma_N = \{0\}$ and $\bar{q} = \partial_n u(0) = 1$, hence

$$F_i^{\text{Neu}} = \bar{q}\, \varphi_i(0).$$

Only $\varphi_1(0) = 1$ is nonzero, so

$$\mathbf{f} = \mathbf{f}^{\text{vol}} + \mathbf{f}^{\text{Neu}} = \begin{bmatrix} \frac{9}{8} \\ \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{8} \end{bmatrix}.$$

Block decomposition and reduced system. We impose the Dirichlet condition at node 5:

$$I_D = \{5\}, \qquad I_F = \{1, 2, 3, 4\}, \qquad U_5 = 1.$$

With the partition $\mathbf{U} = [\mathbf{U}_F; \mathbf{U}_D]$, the system $\mathbf{KU} = \mathbf{F}$ reads

$$\begin{bmatrix} K_{FF} & K_{FD} \\ K_{DF} & K_{DD} \end{bmatrix} \begin{bmatrix} \mathbf{u}_F \\ \mathbf{u}_D \end{bmatrix} = \begin{bmatrix} \mathbf{f}_F \\ \mathbf{f}_D \end{bmatrix}, \qquad \mathbf{U}_D = [U_5] = [1].$$

Extracting the blocks gives

$$K_{FF} = \begin{bmatrix} 4 & -4 & 0 & 0 \\ -4 & 8 & -4 & 0 \\ 0 & -4 & 8 & -4 \\ 0 & 0 & -4 & 8 \end{bmatrix}, \qquad K_{FD} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ -4 \end{bmatrix}, \qquad \mathbf{f}_F = \begin{bmatrix} \frac{9}{8} \\ \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \end{bmatrix}.$$

The reduced system is

$$\boxed{K_{FF}\mathbf{u}_F = \mathbf{f}_F - K_{FD}\mathbf{u}_D.}$$

Since $K_{FD}\mathbf{u}_D = [0, 0, 0, -4]^T$, we obtain the modified right-hand side

$$\mathbf{f}_F - K_{FD}\mathbf{u}_D = \begin{bmatrix} \frac{9}{8} \\ \frac{1}{4} \\ \frac{1}{4} \\ \frac{17}{4} \end{bmatrix}.$$

Solution. Solving the 4×4 reduced system yields

$$u_1 = \frac{5}{2}, \qquad u_2 = \frac{71}{32}, \qquad u_3 = \frac{15}{8}, \qquad u_4 = \frac{47}{32}, \qquad u_5 = 1,$$

so the full nodal vector is

$$\boxed{\mathbf{U} = \begin{bmatrix} \frac{5}{2} \\ \frac{71}{32} \\ \frac{15}{8} \\ \frac{47}{32} \\ 1 \end{bmatrix}.}$$

These nodal values coincide with the exact solution evaluated at the mesh nodes $x_i \in \{0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1\}$.

Bibliography

[1] Jean-Marie Raviart. Finite element approximation of the navier–stokes equations. In Philippe G. Ciarlet and Jacques-Louis Lions, editors, *Numerical Analysis*, volume 2 of *Studies in Mathematics and its Applications*, pages 1–78. North-Holland, 1978.

[2] Gilbert Strang and George J. Fix. *An Analysis of the Finite Element Method*, volume 12 of *Classics in Applied Mathematics*. SIAM, 2008.